

Question:

Write a JAVA program (**LibrarySystem**) to demonstrate the core Object-Oriented Programming concepts: classes and objects, constructors (default and parameterized), encapsulation, inheritance, polymorphism (method overloading and method overriding), abstraction (abstract class) and interfaces. Create the required classes (Book, Person, Student, Faculty, AreaCalculator, Vehicle, Car, Bike, Shape, Circle, Rectangle, Printable, Report) and assume the required member variables and methods in each class.

Execution Steps:

Step 1: Program execution starts from main().

Step 2: " Book Details : " is displayed.

Step 3: Book object b1 is created using the default constructor: bookId = 0, bookName = "Unknown", author = "Unknown", price = 0.0.

Step 4: Book object b2 is created using the parameterized constructor with values 235, "Java programming practices", "Santosh Kumar P", 499.99.

Step 5: b1.displayDetails() and b2.displayDetails() print the details of both books.

Step 6: Student object s is created and name, age and course are set. s.display() calls Person's display() through super.display(), then prints the course.

Step 7: Faculty object f is created and name, age and department are set. f.display() prints the name and age, then the department.

Step 8: AreaCalculator object ac is created. The overloaded area() methods are called: area(5) for the circle, area(4, 6) for the rectangle and area(3, 7, true) for the triangle. The compiler selects the method by the number and types of arguments (compile-time polymorphism).

Step 9: Vehicle references v, car and bike are created pointing to Vehicle, Car and Bike objects. v.display(), car.display() and bike.display() call the overridden version of the actual object (run-time polymorphism).

Step 10: Shape references circle and rectangle are created pointing to Circle and Rectangle objects. circle.draw() and rectangle.draw() run the implementations of the abstract method draw().

Step 11: Report object report is created and report.print() runs the implementation of the Printable interface method.

Step 12: main() ends and the program terminates.

Algorithm:

Step 1: Start.

Step 2: Create the classes Book, Person, Student, Faculty, AreaCalculator, Vehicle, Car, Bike, Shape, Circle, Rectangle, Printable and Report.

Step 3: Create Book objects b1 (default constructor) and b2 (parameterized constructor) and display their details.

Step 4: Create Student and Faculty objects, set their member variables and call display() (inheritance).

Step 5: Create an AreaCalculator object and call area() with one, two and three arguments (method overloading).

Step 6: Create Vehicle references pointing to Vehicle, Car and Bike objects and call display() (method overriding).

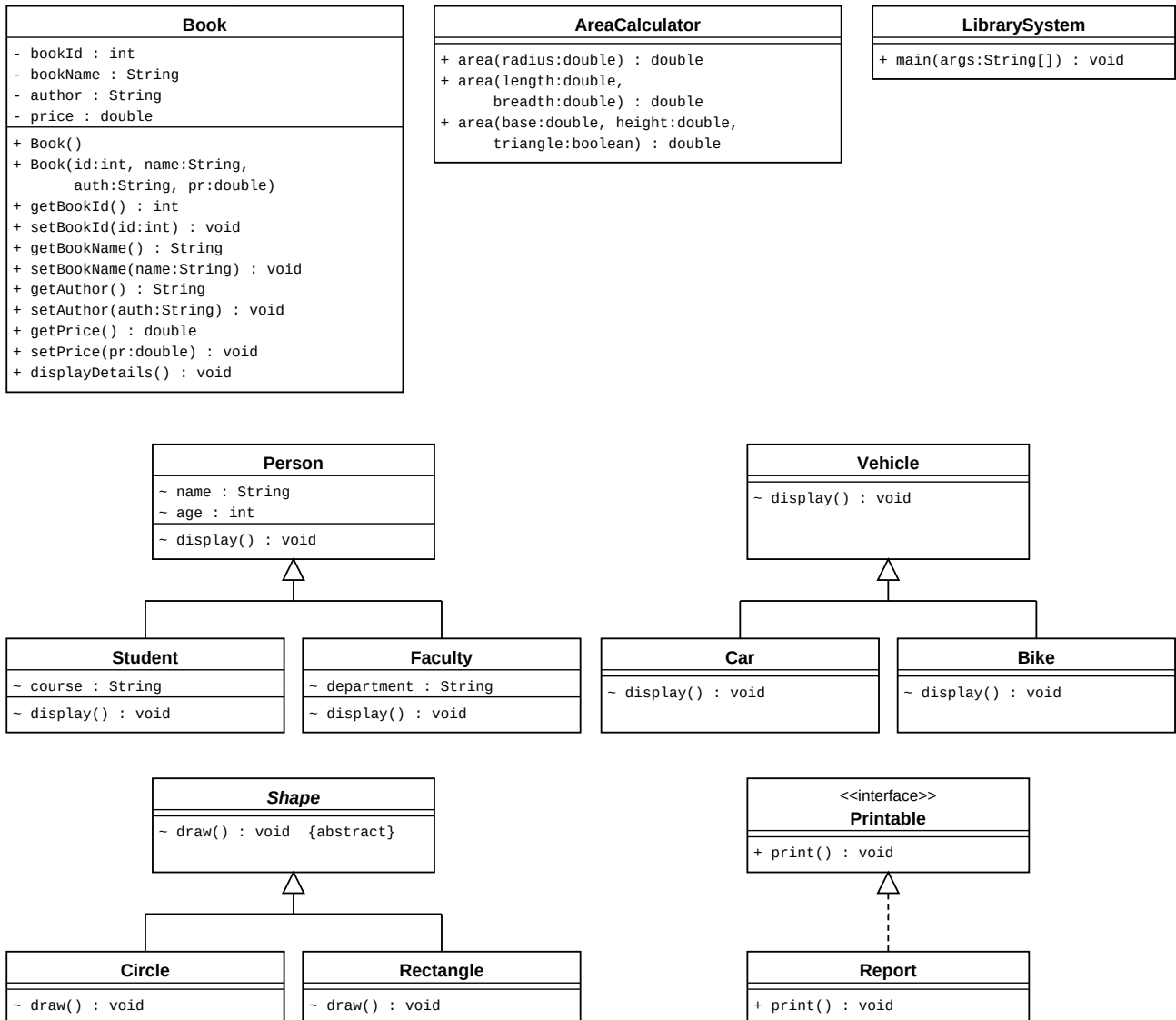
Step 7: Create Shape references pointing to Circle and Rectangle objects and call draw() (abstraction).

Step 8: Create a Report object and call print() (interface).

Step 9: Display all the results.

Step 10: Stop.

UML Class Diagram:



Hollow triangle with a solid line = inheritance (extends); hollow triangle with a dashed line = implementation (implements). Italic name = abstract class. '+' public, '-' private, '~' default (package) access.

Code:

```
package librarysystem;
class Book {
    private int bookId;
    private String bookName;
    private String author;
    private double price;

    // const
    Book() {
        bookId = 0;
        bookName = "Unknown";
        author = "Unknown";
        price = 0.0;
    }
    //const with param
    Book(int id, String name, String auth, double pr) {
        bookId = id;
        bookName = name;
        author = auth;
        price = pr;
    }

    // getters & setters for Encapsulation
    public int getBookId() { return bookId; }
    public void setBookId(int id) { bookId = id; }

    public String getBookName() { return bookName; }
    public void setBookName(String name) { bookName = name; }

    public String getAuthor() { return author; }
    public void setAuthor(String auth) { author = auth; }

    public double getPrice() { return price; }
    public void setPrice(double pr) { price = pr; }

    public void displayDetails() {
        System.out.println(bookId + " | " + bookName + " | " + author + " | " + price);
    }
}
// inheritance super:person sub:student and faculty
class Person {
    String name;
    int age;
    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

class Student extends Person {
    String course;
    void display() {
        super.display();
        System.out.println("Course: " + course);
    }
}

class Faculty extends Person {
    String department;
    void display() {
        super.display();
        System.out.println("Department: " + department);
    }
}

//polymorph
class AreaCalculator {
    double area(double radius) {
        return Math.PI * radius * radius;
    }
}
```

```

    }
    double area(double length, double breadth) {
        return length * breadth;
    }
    double area(double base, double height, boolean triangle) {
        return 0.5 * base * height;
    }
}

class Vehicle {
    void display() {
        System.out.println("This is a vehicle.");
    }
}

class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Car.");
    }
}

class Bike extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Bike.");
    }
}

// abstraction
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

interface Printable {
    void print();
}

class Report implements Printable {
    public void print() {
        System.out.println("Printing Report...");
    }
}

public class LibrarySystem {
    public static void main(String[] args) {
        System.out.println(" Book Details :");
        Book b1 = new Book();
        Book b2 = new Book(235, "Java programming practices", "Santosh Kumar P", 499.99);
        b1.displayDetails();
        b2.displayDetails();

        System.out.println("\n Inheritance:");
        Student s = new Student();
        s.name = "Haarika";
    }
}

```

```

s.age = 20;
s.course = "Computer Science";
s.display();

Faculty f = new Faculty();
f.name = "Dr. Rao";
f.age = 45;
f.department = "Engineering";
f.display();

System.out.println("\nPolymorphism:");
AreaCalculator ac = new AreaCalculator();
System.out.println("Circle Area: " + ac.area(5));
System.out.println("Rectangle Area: " + ac.area(4, 6));
System.out.println("Triangle Area: " + ac.area(3, 7, true));

Vehicle v = new Vehicle();
Vehicle car = new Car();
Vehicle bike = new Bike();
v.display();
car.display();
bike.display();

System.out.println("\n Abstraction & Interfaces");
Shape circle = new Circle();
Shape rectangle = new Rectangle();
circle.draw();
rectangle.draw();
Report report = new Report();
report.print();
}
}

```

Output:

```
Book Details :
0 | Unknown | Unknown | 0.0
235 | Java programming practices | Santosh Kumar P | 499.99

Inheritance:
Name: Haarika, Age: 20
Course: Computer Science
Name: Dr. Rao, Age: 45
Department: Engineering

Polymorphism:
Circle Area: 78.53981633974483
Rectangle Area: 24.0
Triangle Area: 10.5
This is a vehicle.
This is a Car.
This is a Bike.

Abstraction & Interfaces
Drawing a Circle
Drawing a Rectangle
Printing Report...
```

Observation Questions:

Question 1: What is Object-Oriented Programming (OOP)?

OOP is a way of writing programs using objects. Objects have data (variables) and actions (methods). It makes programs easier to manage and reuse.

Question 2: Difference between Class and Object.

A class is a design or blueprint. An object is the real thing made from that design.
Example: Book is a class, b1 = new Book() is an object.

Question 3: Purpose of Constructors.

Constructors are used to give starting values to objects, i.e. to initialize variables. They run automatically when an object is created. If no constructor is written, Java provides a default one. A default constructor gives default values, while a parameterized constructor takes values from the user, as seen in the Book class (b1 uses the default constructor, b2 the parameterized one).

Question 4: What is Encapsulation? How is it achieved in Java?

Encapsulation means keeping variables safe inside a class and controlling who can access them. In Java we make the variables private and use getter/setter methods to access them, as in the Book class above.

Question 5: Difference between Method Overloading and Method Overriding.

Overloading: same method name but different parameters (same class).

Overriding: same method name and parameters but different code, written in the child class.

Question 6: Four pillars of OOP with examples.

Encapsulation: Book class with private variables and getters/setters.

Inheritance: Student and Faculty classes extend Person.

Polymorphism: area() method used for circle, rectangle and triangle (overloading) and display() in Vehicle/Car/Bike (overriding).

Abstraction: Shape abstract class with draw() method, implemented in Circle and Rectangle.